Post Quantum Cryptography

Implementation and analysis of PQC algorithm

Aayush Dhakal

Bachelors in Computer Networking and IT Security

Islington College, Nepal

August 2026

Table of Contents

Table of Tables

Table of Figures

1 Introduction to Post Quantum Cryptography

Classical cryptographic algorithms such as RSA involve using math, especially large prime numbers which are very difficult for computers to reverse. Such algorithms are said to be breakable when quantum cryptography comes into existence.

1.1 Asymmetric Cryptography

Asymmetric cryptographic algorithms such as RSA and ECC use prime factorization and discrete logarithms. Prime factorization involves two large prime numbers p and q whose product is s.

$$p.q=s$$

The value of s is known. So, the computer tries to brute-force to get the value of p and q. It takes a very long time (thousand to million years) even for supercomputers. A quantum computer uses Shor's algorithm which can break prime factorization exponentially. So, it takes few hours for quantum computer to break this algorithm.

1.2 Symmetric Cryptography

Symmetric cryptographic algorithms such as AES are more secure against quantum computers than asymmetric cryptographic algorithms. Quantum computers use Grover's Algorithm to halve the AES encryption. So, AES-128 becomes AES 64 and AES-256 becomes AES-128. Since AES-128 is a very secure algorithm, AES-256 is theoretically secure against quantum cryptography. However, key exchange mechanisms are vulnerable to quantum computing in symmetric cryptography.

1.3 Harvest Now, Decrypt Later (HNDL)

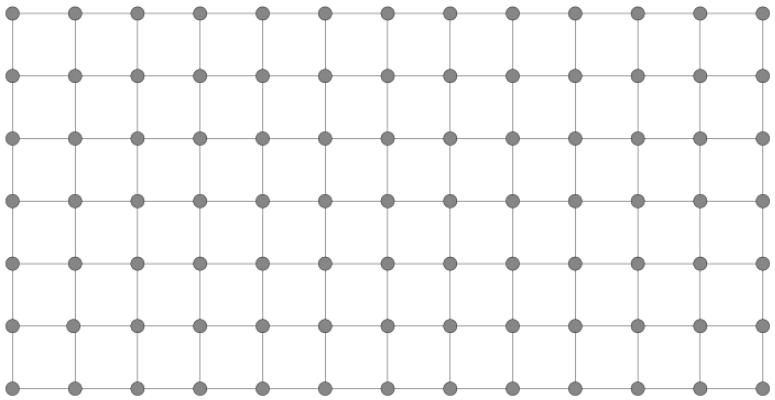
Quantum Computing has not been developed but it will be available in the future. Various state-sponsored attackers and APT groups have intercepted encrypted network traffics and stored them. When quantum computers become available, these attackers will decrypt the traffic and get sensitive information. This is called Harvest Now Decrypt Later (HNDL) where they harvest the encrypted data now and decrypt in the future with

quantum computers. Various personal information may not be useful in the next 20 to 30 years but military information, state secrets, etc. will be very useful for attackers.

1.4 Lattice-Based Cryptography

Prime factorization is useless in quantum computing. Post Quantum Cryptography uses Lattice-Based Cryptography.

A 2-dimensional lattice grid includes dots with intersections. One of the dots is the message encrypted. The mathematical algorithm of cryptography starts at one dot and takes various paths until it finds the shortest path that reaches to the closest dot. A noise is a random mathematical value added to the algorithm that shifts the ciphertext away from the exact dot. Without the private key, finding the original point among the noise is practically impossible.



If the same concept is applied to a lattice with infinite dots and 1000 dimensions, the Post Quantum Cryptographic lattice is obtained. It is difficult for quantum computers to break the algorithms based on lattice-based cryptography.

1.5 NIST Standards

National Institute of Standards and Technology (NIST) has been continuously proposing Post Quantum Cryptographic Algorithms. It has finalized three core standards:

1. ML-KEM (FIPS 203)
2. ML-DSA (FIPS 204)
3. SLH-DSA (FIPS 205)

2 Methodology

The ML-KEM algorithm of NIST was used to demonstrate Post Quantum Cryptography. The program works in three phases.

1. The server creates the public/private key pair.
2. The client uses the public key to generate the shared secret *and* encapsulate it into the ciphertext simultaneously.
3. The server decapsulates the ciphertext to reveal the identical shared secret. The ciphertext is the encrypted key.

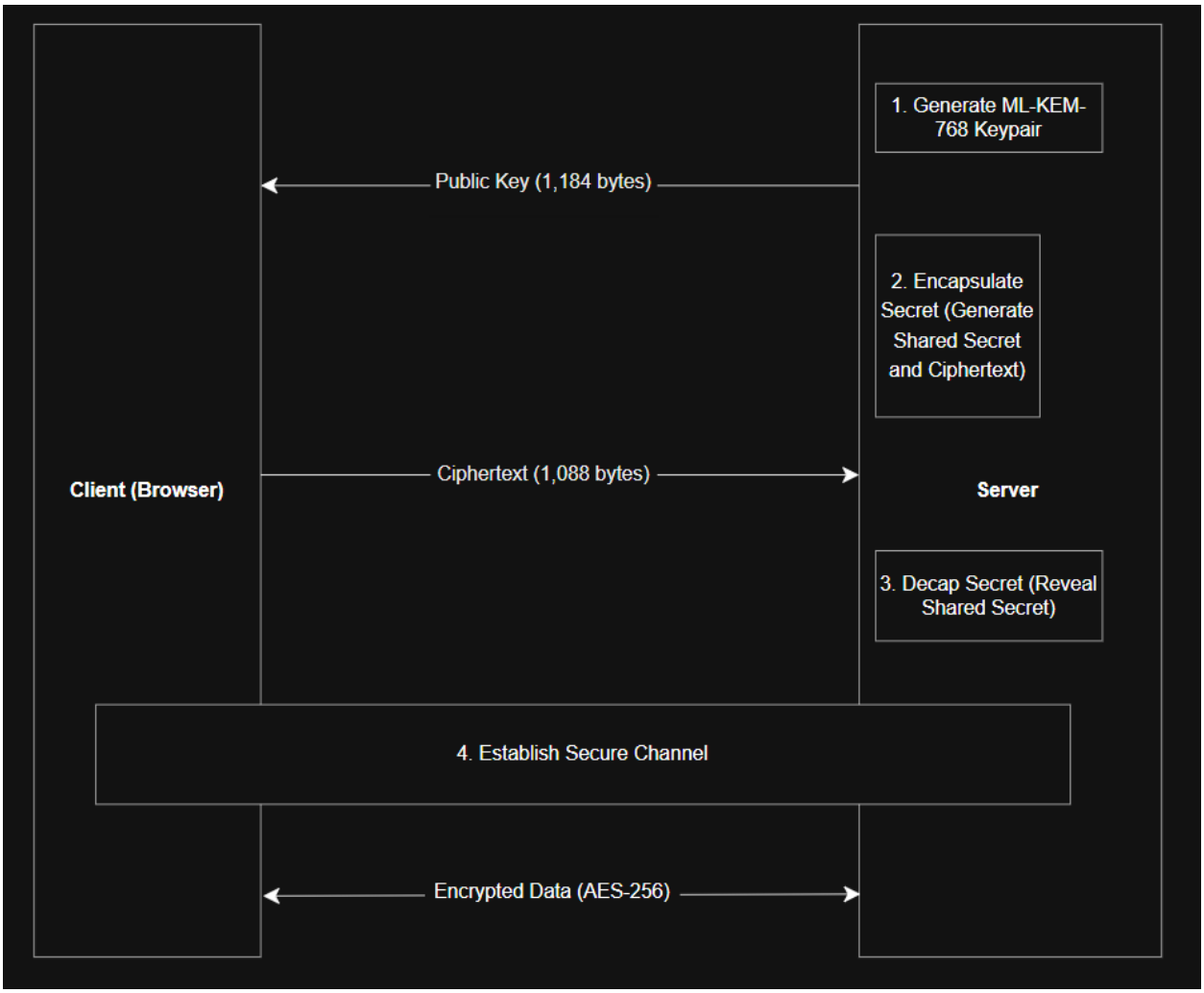


Figure 1 Network Diagram

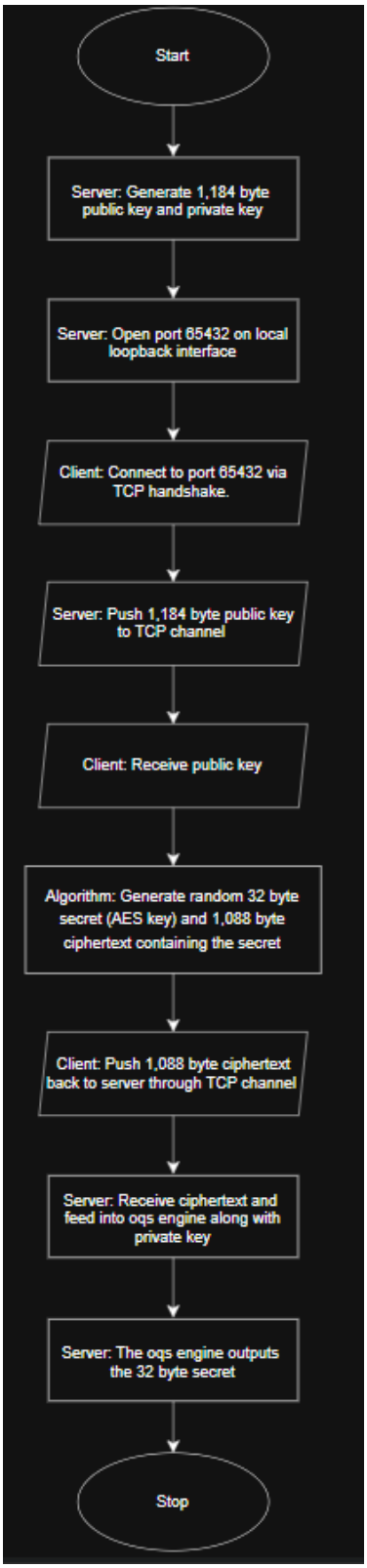


Figure 2 Flowchart

Two scripts client.py and server.py were used to implement Post Quantum Cryptography. The scripts have two libraries: sockets and oqs. A socket is a two-way communication channel across a network. Open Quantum Safe (OQS) is an open-source project of Post Quantum Cryptography.

When the scripts are run, server.py should open a network port and wait for the connection. When client.py is run, it should connect to the server. The server should then send the massive public key to the client, which the client should use to encapsulate a secret and send the ciphertext back. The packets can be captured by tcpdump and analyzed in Wireshark.

3 Execution

1. The mlkem_env was activated.

   source mlkem_env/bin/activate

Figure 3 Activating the environment

2. Packet capture on loopback interface at port 65432 was started in the terminal via tcpdump.

   sudo tcpdump -i lo port 65432 -w pqc_capture.pcap

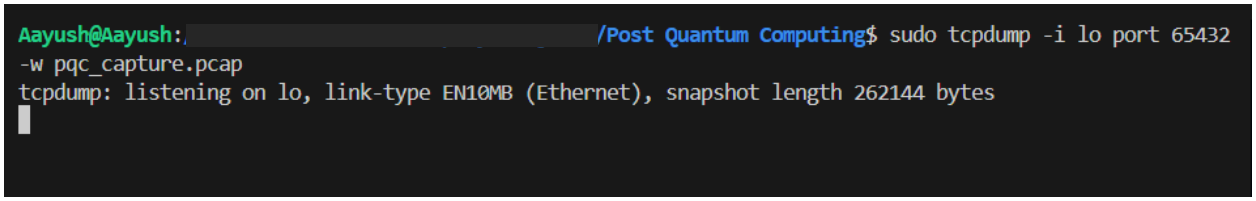


Figure 4 Capturing packets using tcpdump

3. The two scripts were run in another two terminals.

   python3 server.py

   python3 client.py

```
(mlkem_env) Aayush@Aayush:                                    /Post Quantum Computing$ python3 server.py
liboqs-python faulthandler is disabled
Generating ML-KEM-768 Keypair...
Server listening on 127.0.0.1:65432
```

```
(mlkem_env) Aayush@Aayush:                                    /Post Quantum Computing$ python3 client.py
liboqs-python faulthandler is disabled
Connecting to Server at 127.0.0.1:65432...
Waiting for Server Public Key...
Received Public Key (1184 bytes).
Sending Ciphertext (1088 bytes) over TCP...

Secret encapsulated and sent. Ready for AES-256 data transmission.
(mlkem_env) Aayush@Aayush:                                    /Post Quantum Computing$
```

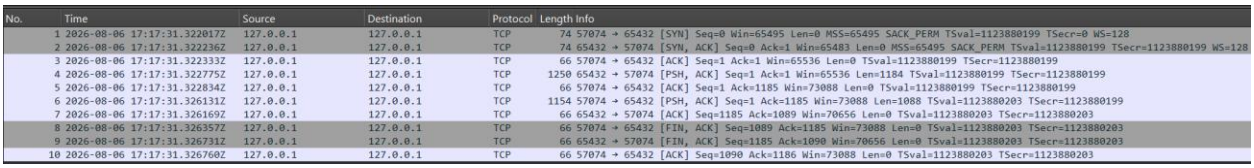
Figure 5 Running scripts

4. The packet capture was stopped and the pcap file generated, which was opened with Wireshark.

```
(mlkem_env) Aayush@Aayush:                                   'Post Quantum Computing$ sudo tcpdump -i lo
 port 65432 -w pqc_capture.pcap
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C10 packets captured
20 packets received by filter
0 packets dropped by kernel
(mlkem_env) Aayush@Aayush:                                   'Post Quantum Computing$
```

Figure 6 Stopping the packet capture

4 Analysis



No.	Time	Source	Destination	Protocol	Length	Info
1	2026-08-06 17:17:31.322017Z	127.0.0.1	127.0.0.1	TCP	74	57074 → 65432 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=1123880199 TSecr=0 WS=128
2	2026-08-06 17:17:31.322236Z	127.0.0.1	127.0.0.1	TCP	74	65432 → 57074 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=1123880199 TSecr=1123880199 WS=128
3	2026-08-06 17:17:31.322333Z	127.0.0.1	127.0.0.1	TCP	66	57074 → 65432 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=1123880199 TSecr=1123880199
4	2026-08-06 17:17:31.322775Z	127.0.0.1	127.0.0.1	TCP	1250	65432 → 57074 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=1184 TSval=1123880199 TSecr=1123880199
5	2026-08-06 17:17:31.322834Z	127.0.0.1	127.0.0.1	TCP	66	57074 → 65432 [ACK] Seq=1 Ack=1185 Win=73088 Len=0 TSval=1123880199 TSecr=1123880199
6	2026-08-06 17:17:31.326131Z	127.0.0.1	127.0.0.1	TCP	1154	57074 → 65432 [PSH, ACK] Seq=1 Ack=1185 Win=73088 Len=1088 TSval=1123880203 TSecr=1123880199
7	2026-08-06 17:17:31.326169Z	127.0.0.1	127.0.0.1	TCP	66	65432 → 57074 [ACK] Seq=1185 Ack=1089 Win=70656 Len=0 TSval=1123880203 TSecr=1123880203
8	2026-08-06 17:17:31.326357Z	127.0.0.1	127.0.0.1	TCP	66	57074 → 65432 [FIN, ACK] Seq=1089 Ack=1185 Win=73088 Len=0 TSval=1123880203 TSecr=1123880203
9	2026-08-06 17:17:31.326731Z	127.0.0.1	127.0.0.1	TCP	66	65432 → 57074 [FIN, ACK] Seq=1185 Ack=1090 Win=70656 Len=0 TSval=1123880203 TSecr=1123880203
10	2026-08-06 17:17:31.326760Z	127.0.0.1	127.0.0.1	TCP	66	57074 → 65432 [ACK] Seq=1090 Ack=1186 Win=73088 Len=0 TSval=1123880203 TSecr=1123880203

Figure 7 Captured Packets in Wireshark

The server's port is 65432 and the client's port is 57074. 10 packets had been captured in the pcap file.

Packet No.	Analysis	Payload Length
1	Client sent SYN packet to the server.	0
2	Server sent SYN-ACK packet to the client.	0
3	Client sent ACK packet to the server establishing TCP connection.	0
4	Server pushes data to the client. This is the ML-KEM-768 Public Key.	1184
5	Client acknowledges that it had received the data.	0
6	Client pushes data to the server. This is the ciphertext.	1088
7	Server acknowledges that it had received the data.	0
8-10	The TCP connection is terminated.	0

Table 1 Analysis Table

The server had sent 1,184 bytes of public key to the client. AES is a symmetric encryption method. However, ML-KEM is asymmetric. So, the server also has a private key.

The client feeds the public key to the ML-KEM algorithm. This generates a 32 byte AES key which is simultaneously encrypted into 1,088 byte ciphertext. The ciphertext is sent to the server.

The server feeds the 1,088 byte ciphertext and the private key into the ML-KEM algorithm and it outputs the 32 byte AES key. The key exchange process ends.

5 Conclusion

AES is a symmetric encryption method. Even though the encryption algorithm is resistant to quantum computing, the delivery of the key from one party to another is the problem with AES. Combining ML-KEM with AES helps to solve this problem. However, in the packets analysis with Wireshark, the packets have significant network overhead. The 1,184 bytes of public key is huge for a standard packet and is close to 1,500 bytes Maximum Transmission Unit (MTU). This could result in packet fragmentation.